

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title: 1) Set up the Node.js environment for the Book Library Manager backend and display a basic server response

Page No:

AIM: Set up the Node.js environment for the Book Library Manager backend and display a basic server response.

Description: This experiment demonstrates the basic setup of a Node.js environment using the Book Library Manager backend application. The Express server is initialized and a simple root route is created to verify successful configuration. When the server runs, it returns a basic text response confirming that the backend environment is installed, connected, and ready for further development.

Source Code:

File: backend/server.js

```
require("dotenv").config();
const express = require("express");
const cors = require("cors");
const connectDB = require("../src/config/db");
const userRoutes =
  require("../src/routes/userRoutes");
const bookRoutes =
  require("../src/routes/bookRoutes");
const authRoutes =
  require("../src/routes/authRoutes");

const app = express();
app.use(cors());
app.use(express.json());

app.use("/api/users", userRoutes);
app.use("/api/books", bookRoutes);
app.use("/api/auth", authRoutes);

app.get("/", (req, res) => {
  res.send("Book Library Manager Backend Running");
});

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

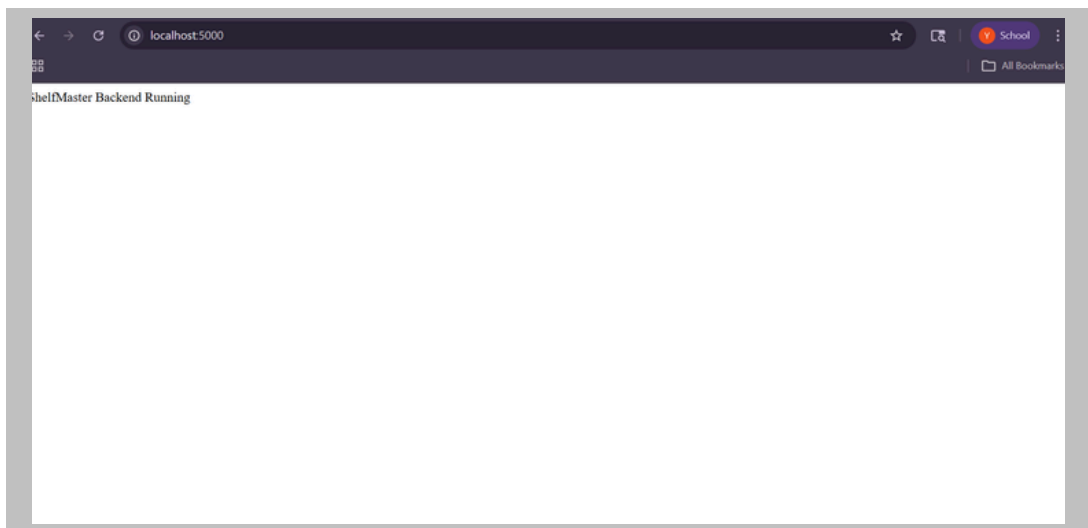
Date:

Title:

Page No:

OUTPUT:

```
Run `npm audit` for details.
PS C:\Users\Yasaswini\OneDrive\Desktop\Book_Library_Manager\backend> node server.js
[dotenv@17.3.1] injecting env (3) from .env -- tip: ⚙ override existing env vars with
  { override: true }
Server running on port 5000
MongoDB Connected
```



BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title: 2) Develop a Node.js based user login system for the Book Library Manager application

Page No:

AIM: Develop a Node.js based user login system for the Book Library Manager application

Description: This experiment implements a user login system using Node.js, Express, and MongoDB. It validates user credentials, checks encrypted passwords with bcrypt, and generates a JWT token after successful login. The Book Library Manager project uses this module for secure authentication of users and admins. It shows how backend login logic can be used to control access to protected application features.

Source Code:

File: backend/src/routes/authRoutes.js

```
const express = require("express");
const {
  register,
  registerAdmin,
  login,
  resetPassword,
  me
} = require("../controllers/authController");
const { protect } = require("../middleware/authMiddleware");
const router = express.Router();
router.post("/register", register);
router.post("/register-admin", registerAdmin);
router.post("/login", login);
router.post("/reset-password", resetPassword);
router.get("/me", protect, me);
module.exports = router;
```

File: backend/src/controllers/authController.js

```
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const User = require("../models/User");
const buildToken = (user) =>
  jwt.sign(
    { id: user._id, role: user.role },
    process.env.JWT_SECRET || "secretkey",
    { expiresIn: "1d" }
  );

const sanitizeUser = (user) => {
  const { password, ...userData } = user._doc;
  return userData;
};
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

```
exports.login = async (req, res) => {
  try {
    const { email, password } = req.body;
    if (!email || !password) {
      return res.status(400).json({ message: "Email and password are required" });
    }
    const normalizedEmail = email.trim().toLowerCase();
    const user = await User.findOne({ email: normalizedEmail });
    if (!user) {
      return res.status(400).json({ message: "User not found" });
    }
    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) {
      return res.status(400).json({ message: "Invalid password" });
    }
    res.json({
      token: buildToken(user),
      user: sanitizeUser(user)
    });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};
```

File: backend/src/models/User.js

```
const mongoose = require("mongoose");
const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      trim: true
    },
    email: {
      type: String,
      required: true,
      unique: true,
      lowercase: true,
      trim: true
    },
  },
  {
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

```
role: {
  type: String,
  enum: ["user", "admin"],
  required: true
},
password: {
  type: String,
  required: true,
  minlength: 6
}
{ timestamps: true }
);
module.exports = mongoose.model("User", userSchema);
```

OUTPUT:

POST http://localhost:5000/api/auth/register Send

Status: 200 OK Size: 254 Bytes Time: 125 ms

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "email": "user@example.com",
3   "password": "password123",
4   "name": "Test User",
5   "role": "user"
6 }
```

Response Headers 7 Cookies Results Docs </>

```
1 {
2   "name": "Test User",
3   "email": "user@example.com",
4   "password": "$2b$10$W0GiFEGkV2iq07WoyI4d
5     .4c9W6H43Ld6uWzJ5G2O7BvqlgCKVaEq",
6   "role": "user",
7   "_id": "69db480f558ce893c329cb52",
8   "createdAt": "2026-04-12T07:21:51.951Z",
9   "updatedAt": "2026-04-12T07:21:51.951Z",
10  "__v": 0
11 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Yasaswini\OneDrive\Desktop\Book_Library_Man

To address all issues, run:

```
npm audit fix
```

Run 'npm audit' for details.

PS C:\Users\Yasaswini\OneDrive\Desktop\Book_Library_Manager\backend> node server.js

[dotenv@17.3.1] injecting env (3) from .env -- tip: override existing env vars with { override: true }

Server running on port 5000

MongoDB Connected

no... wsl

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title: 3)Write a Node.js program to insert and remove Book Library Manager project data using backend scripts

Page No:

AIM: Write a Node.js program to insert and remove Book Library Manager project data using backend scripts

Description:

This experiment focuses on performing data operations similar to read, write, and delete actions in Node.js. In Book Library Manager, these actions are handled through MongoDB scripts rather than traditional file system operations. The seed and reset scripts insert sample records and remove existing records from the database. It demonstrates programmatic data management for application setup and testing.

Source Code:

File : backend/src/seed.js

```
require("dotenv").config();
const bcrypt = require("bcryptjs");
const connectDB = require("../config/db");
const User = require("../models/User");
const Category = require("../models/Category");
const Book = require("../models/Book");

connectDB();

const seed = async () => {
  await User.deleteMany();
  await Category.deleteMany();
  await Book.deleteMany();

  const password = await bcrypt.hash("Password123", 10);

  await User.create([
    {
      name: "Admin User",
      email: "admin@library.com",
      role: "admin",
      password
    },
    {
      name: "Regular User",
      email: "user@library.com",
      role: "user",
      password
    }
  ]);
};
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

```
const categories = await Category.create([
  { name: "Fiction", description: "Fictional literature" },
  { name: "Science", description: "Scientific books" },
  { name: "History", description: "Historical books" }
]);
await Book.create([
  {
    title: "The Great Gatsby",
    author: "F. Scott Fitzgerald",
    category: categories[0]._id,
    description: "A classic American novel",
    publishedYear: 1925,
    isbn: "9780743273565"
  }
]);
console.log("Seed data inserted");
console.log("Default password for all seeded users: Password123");
process.exit();
};
```

seed();

File: backend/src/resetData.js

```
require("dotenv").config();
const connectDB = require("../config/db");
const User = require("../models/User");
const Book = require("../models/Book");
const Category = require("../models/Category");
const ReadingList = require("../models/ReadingList");

connectDB();

const resetData = async () => {
  await ReadingList.deleteMany();
  await Book.deleteMany();
  await Category.deleteMany();
  await User.deleteMany();
  console.log("All users, books, categories and reading lists have
been removed.");
  process.exit();
};
resetData();
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

OUTPUT:

+ Insert Document

Refresh501-49 of 49

▼{
_id: ObjectId('69db48d0353896a3f13008e1'),
title: 'Harry Potter and the Sorcerer's Stone',
author: 'J.K. Rowling',
description: 'A boy discovers he is a wizard and begins his journey at Hogwarts.',
genre: 'Fantasy',
__v: NumberInt('0'),
createdAt: ISODate('2026-04-12T07:25:04.421Z'),
updatedAt: ISODate('2026-04-12T07:25:04.421Z')
}

▼{
_id: ObjectId('69db48d0353896a3f13008e2'),
title: 'Harry Potter and the Chamber of Secrets',
author: 'J.K. Rowling',
description: 'A dark force threatens Hogwarts as secrets from the past emerge.'
}

PROBLEMSOUTPUTDEBUG CONSOLETERMINALPORTS

To address all issues, run:
npm audit fix

Run `npm audit` for details.
PS C:\Users\Yasaswini\OneDrive\Desktop\Book_Library_Manager\backend> node server.js
[dotenv@17.3.1] injecting env (3) from .env -- tip: ⚙️ override existing env vars with { override: true }
Server running on port 5000
MongoDB Connected

PS C:\Users\Yasaswini\OneDrive\Desktop\Book_Library_Manager\bac
kend> node seedBooks.js
[dotenv@17.3.1] injecting env (3) from .env -- tip: 🔒 prevent
building .env in docker: https://dotenvx.com/prebuild
Books Inserted ✓
PS C:\Users\Yasaswini\OneDrive\Desktop\Book_Library_Manager\bac
kend>

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title: 4)Implement HTTP request and response handling for the Book Library Manager book management API.

Page No:

AIM: Implement HTTP request and response handling for the Book Library Manager book management API.

Description: This experiment shows how Node.js handles HTTP requests and responses through backend routes. In Book Library Manager, book operations are submitted through API endpoints created with Express. The server receives request data, validates it, processes it, and returns a JSON response. It highlights the request-response cycle used in modern web application backends.

Source Code:

File: backend/src/routes/bookRoutes.js

```
const express = require("express");
const {
  getAllBooks,
  getBookById,
  createBook,
  updateBook,
  deleteBook
} = require("../controllers/bookController");
const { protect, authorize } = require("../middleware/authMiddleware");
const router = express.Router();

router.get("/", getAllBooks);
router.get("/:id", getBookById);
router.post("/", protect, authorize("admin"), createBook);
router.put("/:id", protect, authorize("admin"), updateBook);
router.delete("/:id", protect, authorize("admin"), deleteBook);
module.exports = router;
```

File: backend/src/controllers/bookController.js

```
const Book = require("../models/Book");
exports.getAllBooks = async (req, res) => {
  try {
    const { search, category } = req.query;
    let query = {};
    if (search) {
      query.$or = [
        { title: { $regex: search, $options: "i" } },
        { author: { $regex: search, $options: "i" } }
      ];
    }
  }
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

```
if (category) {
  query.category = category;
}

const books = await Book.find(query)
  .populate("category", "name")
  .sort({ createdAt: -1 });

res.json(books);
} catch (error) {
  res.status(500).json({ message: error.message });
}
};

exports.createBook = async (req, res) => {
  try {
    const { title, author, category, description, publishedYear, isbn } = req.body;

    if (!title || !author || !category) {
      return res.status(400).json({ message: "Title, author and category are required" });
    }

    const book = await Book.create({
      title: title.trim(),
      author: author.trim(),
      category,
      description: description?.trim() || "",
      publishedYear,
      isbn: isbn?.trim() || ""
    });

    const populatedBook = await Book.findById(book._id).populate("category", "name");
    res.status(201).json(populatedBook);
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
};
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

OUTPUT:

GET

Status: 200 OK Size: 12.53 KB Time: 24 ms

Query Headers² Auth **Body** Tests Pre Run

JSON XML Text Form Form-encode GraphQL

JSON Content

```
1 [
2   {
3     "_id": "69db48d0353896a3f13008e1",
4     "title": "Harry Potter and the Sorcerer's Stone",
5     "author": "J.K. Rowling",
6     "description": "A boy discovers he is a wizard and begins his
7       journey at Hogwarts.",
8     "genre": "Fantasy",
9     "__v": 0,
10    "createdAt": "2026-04-12T07:25:04.421Z",
11    "updatedAt": "2026-04-12T07:25:04.421Z"
12  },
13  {
14    "_id": "69db48d0353896a3f13008e2",
15    "title": "Harry Potter and the Chamber of Secrets",
16    "author": "J.K. Rowling",
```

Response

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
ary_Manager\backend> node server.js
[dotenv@17.3.1] injecting env (3) from .env -- tip: override existing env vars with { override: true }
Server running on port 5000
MongoDB Connected
```

POST

Status: 201 Created Size: 528 Bytes Time: 19 ms

Query Headers² Auth **Body¹** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

```
1 {
2   "title": "The Fault in Our Stars",
3   "author": "John Green",
4   "genre": "Young Adult",
5   "description": "Despite the tumor-shrinking medical miracle that has bought her a few
6     years, Hazel has never been anything but terminal, her final chapter inscribed upon
7     diagnosis. But when a gorgeous plot twist named Augustus Waters suddenly appears at
8     Cancer Kid Support Group, Hazel's story is about to be completely rewritten."
9 }
10
```

Response

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
ary_Manager\backend> node server.js
[dotenv@17.3.1] injecting env (3) from .env -- tip: override existing env vars with { override: true }
Server running on port 5000
MongoDB Connected
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title: 5) Connect the Book Library Manager project to MongoDB and define the required database collections.

Page No:

AIM: Connect the Book Library Manager project to MongoDB and define the required database collections.

Description: This experiment demonstrates the use of MongoDB databases and collections in the Book Library Manager project. The application connects to MongoDB and defines schemas for users, books, categories, and reading lists using Mongoose. These schemas represent structured collections that store authentication and library information. It explains how database connection and collection design support the project's data layer.

Source Code:

File: backend/src/config/db.js

```
const mongoose = require("mongoose");
const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("MongoDB Connected");
  } catch (error) {
    console.error("DB Connection Failed:", error.message);
    process.exit(1);
  }
};
module.exports = connectDB;
```

File: backend/src/models/Book.js

```
const mongoose = require("mongoose");

const bookSchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: true,
      trim: true
    },
    author: {
      type: String,
      required: true,
      trim: true
    },
    category: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "Category",
      required: true
    },
  },
  { timestamps: true }
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

```
description: {
  type: String,
  trim: true,
  default: ""
},
publishedYear: {
  type: Number
},
isbn: {
  type: String,
  trim: true
}
},
{ timestamps: true }
);
```

```
module.exports = mongoose.model("Book", bookSchema);
```

File: backend/src/models/ReadingList.js

```
const mongoose = require("mongoose");

const readingListSchema = new mongoose.Schema({
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true
  },
  book: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Book",
    required: true
  },
  addedAt: {
    type: Date,
    default: Date.now
  }
});

readingListSchema.index({ user: 1, book: 1 }, { unique: true });

module.exports = mongoose.model("ReadingList", readingListSchema);
```

BVRIT HYDERABAD College of Engineering for Women

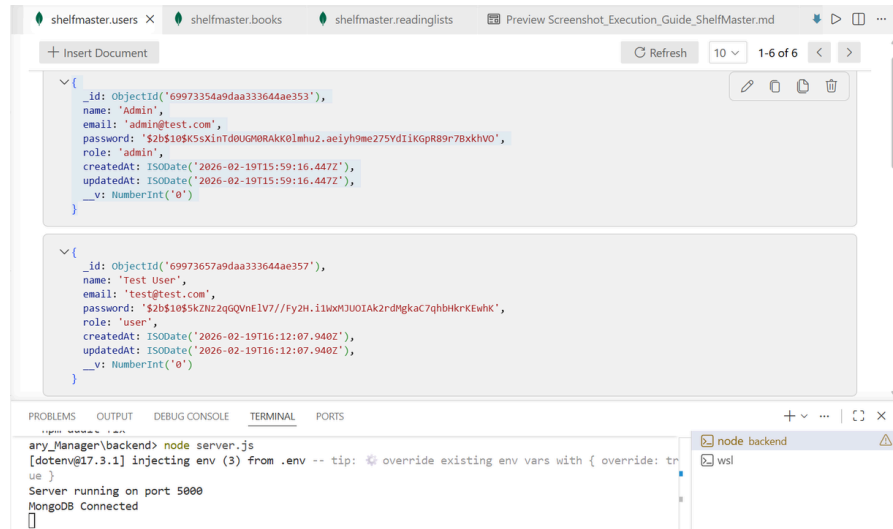
RollNo: 23WH1A0536

Date:

Title:

Page No:

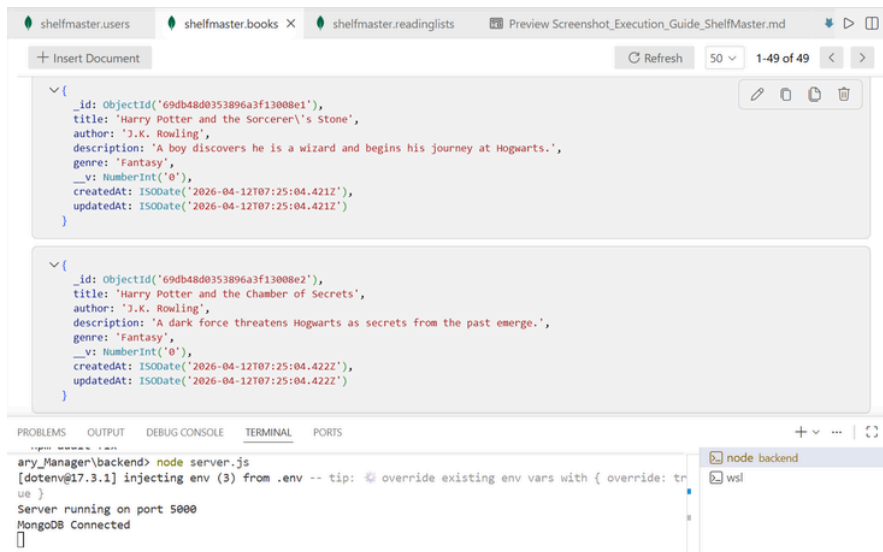
OUTPUT:



```
shelfmaster.users x shelfmaster.books shelfmaster.readinglists Preview Screenshot_Execution_Guide_ShelfMaster.md
+ Insert Document Refresh 10 v 1-6 of 6 < >
{
  _id: ObjectId('69973354a9daa333644ae353'),
  name: 'Admin',
  email: 'admin@test.com',
  password: '$2b$10$K5sXinT8UGH8RAKk0lhu2.aeiyh9me275yDIIGpR89r7BkxhV0',
  role: 'admin',
  createdAt: ISODate('2026-02-19T15:59:16.447Z'),
  updatedAt: ISODate('2026-02-19T15:59:16.447Z'),
  __v: NumberInt('0')
}

{
  _id: ObjectId('69973657a9daa333644ae357'),
  name: 'Test User',
  email: 'test@test.com',
  password: '$2b$10$5K2Nz2qGQVnELV7//Fy2H.11WxH3U0IAK2rdMgkac7qhbHkrKEwhk',
  role: 'user',
  createdAt: ISODate('2026-02-19T16:12:07.940Z'),
  updatedAt: ISODate('2026-02-19T16:12:07.940Z'),
  __v: NumberInt('0')
}

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
ary_Manager\backend> node server.js
[dotenv@17.3.1] injecting env (3) from .env -- tip: override existing env vars with { override: tr
ue }
Server running on port 5000
MongoDB Connected
```



```
shelfmaster.users shelfmaster.books x shelfmaster.readinglists Preview Screenshot_Execution_Guide_ShelfMaster.md
+ Insert Document Refresh 50 v 1-49 of 49 < >
{
  _id: ObjectId('69db48d035896a3f13008e1'),
  title: 'Harry Potter and the Sorcerer's Stone',
  author: 'J.K. Rowling',
  description: 'A boy discovers he is a wizard and begins his journey at Hogwarts.',
  genre: 'Fantasy',
  __v: NumberInt('0'),
  createdAt: ISODate('2026-04-12T07:25:04.421Z'),
  updatedAt: ISODate('2026-04-12T07:25:04.421Z')
}

{
  _id: ObjectId('69db48d035896a3f13008e2'),
  title: 'Harry Potter and the Chamber of Secrets',
  author: 'J.K. Rowling',
  description: 'A dark force threatens Hogwarts as secrets from the past emerge.',
  genre: 'Fantasy',
  __v: NumberInt('0'),
  createdAt: ISODate('2026-04-12T07:25:04.422Z'),
  updatedAt: ISODate('2026-04-12T07:25:04.422Z')
}

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
ary_Manager\backend> node server.js
[dotenv@17.3.1] injecting env (3) from .env -- tip: override existing env vars with { override: tr
ue }
Server running on port 5000
MongoDB Connected
```



```
+ Insert Document Refresh 10 v 1-4 of 4
{
  _id: ObjectId('69974ba7e64e863874d03afb'),
  user: ObjectId('69973354a9daa333644ae353'),
  books: [],
  createdAt: ISODate('2026-02-19T17:43:03.591Z'),
  updatedAt: ISODate('2026-02-19T17:45:19.582Z'),
  __v: NumberInt('2')
}

{
  _id: ObjectId('69d6984ad065d69158c3522a'),
  user: ObjectId('69d69839d065d69158c35222'),
  books: [ ... ],
  createdAt: ISODate('2026-04-08T18:02:50.395Z'),
  updatedAt: ISODate('2026-04-11T07:00:48.783Z'),
  __v: NumberInt('8')
}

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
npm audit fix
ary_Manager\backend> node server.js
[dotenv@17.3.1] injecting env (3) from .env -- tip: override existing env vars with { override: tr
ue }
Server running on port 5000
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title: 6) Implement CRUD operations for Book Library Manager users and book records using MongoDB.

Page No:

AIM: Implement CRUD operations for Book Library Manager users and book records using MongoDB.

Description:

This experiment implements CRUD operations using MongoDB in the Book Library Manager system. Users can be registered, book records can be created, records can be viewed, and book information can be updated. Administrative actions and reset scripts also support deletion or clearing of stored data. It clearly demonstrates the complete lifecycle of data management in a real project.

Source Code:

File: backend/src/controllers/authController.js

```
const createUser = async ({ name, email, password, role }) => {
  const normalizedEmail = email.trim().toLowerCase();
  const existingUser = await User.findOne({ email: normalizedEmail });

  if (existingUser) {
    return { error: { status: 409, message: "User already exists" } };
  }

  const hashedPassword = await bcrypt.hash(password, 10);
  const user = await User.create({
    name: name.trim(),
    email: normalizedEmail,
    password: hashedPassword,
    role
  });

  return { user };
};

exports.register = async (req, res) => {
  try {
    const { name, email, password } = req.body;
    const role = req.body.role || "user";

    if (!name || !email || !password) {
      return res.status(400).json({ message: "Name, email, and password are required" });
    }

    if (!["user", "admin"].includes(role)) {
      return res.status(400).json({ message: "Role must be user or admin" });
    }
  }
}
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

```
const result = await createUser({ name, email, password, role });

if (result.error) {
  return res.status(result.error.status).json({ message: result.error.message });
}

res.status(201).json({
  message: "Registration successful",
  user: sanitizeUser(result.user)
});
} catch (err) {
  res.status(400).json({ message: err.message });
}
};
```

File: backend/src/controllers/bookController.js

```
exports.getBookById = async (req, res) => {
  try {
    const book = await Book.findById(req.params.id)
      .populate("category", "name description");

    if (!book) {
      return res.status(404).json({ message: "Book not found" });
    }

    res.json(book);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

exports.updateBook = async (req, res) => {
  try {
    const { title, author, category, description, publishedYear, isbn } = req.body;

    const book = await Book.findById(req.params.id);
    if (!book) {
      return res.status(404).json({ message: "Book not found" });
    }
  }
};
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

```
book.title = title || book.title;
book.author = author || book.author;
book.category = category || book.category;
book.description = description || book.description;
book.publishedYear = publishedYear || book.publishedYear;
book.isbn = isbn || book.isbn;
```

```
await book.save();
```

```
const populatedBook = await Book.findById(book._id)
.populate("category", "name");
```

```
res.json(populatedBook);
} catch (error) {
res.status(400).json({ message: error.message });
}
};
```

```
exports.deleteBook = async (req, res) => {
try {
const book = await Book.findByIdAndDelete(req.params.id);

if (!book) {
return res.status(404).json({ message: "Book not found" });
}
```

```
res.json({ message: "Book deleted successfully" });
} catch (error) {
res.status(500).json({ message: error.message });
}
};
```

File: backend/src/routes/userRoutes.js

```
const express = require("express");
const User = require("../models/User");
const { protect, authorize } = require("../middleware/authMiddleware");

const router = express.Router();
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

```
router.get("/", protect, authorize("admin"), async (req, res) => {
  try {
    const users = await User.find().select("-password").sort({ createdAt: -1 });
    res.json(users);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});
module.exports = router;
```

OUTPUT:

The screenshot displays the VS Code interface with an API client (Postman) and a terminal window. The API client shows two successful requests to a local server.

Request 1: POST `http://localhost:5000/api/auth/register`. Status: 200 OK, Size: 254 Bytes, Time: 81 ms. The response is a JSON object representing a user:

```
{
  "name": "Test User",
  "email": "test@example.com",
  "password": "$2b$10$k.69HX1MhS2NhBNzLg5tseHEapeHW2BcJBeQDUxcFTdoaAJ0OXcma",
  "role": "user",
  "_id": "69db5094558ce893c329cb5a",
  "createdAt": "2026-04-12T07:58:12.207Z",
  "updatedAt": "2026-04-12T07:58:12.207Z",
  "__v": 0
}
```

Request 2: GET `http://localhost:5000/api/books`. Status: 200 OK, Size: 13.05 KB, Time: 20 ms. The response is a JSON array of book objects:

```
[
  {
    "title": "Harry Potter and the Sorcerer's Stone",
    "author": "J.K. Rowling",
    "description": "A boy discovers he is a wizard and begins his journey at Hogwarts.",
    "genre": "Fantasy",
    "__v": 0,
    "createdAt": "2026-04-12T07:25:04.421Z",
    "updatedAt": "2026-04-12T07:25:04.421Z"
  },
  {
    "_id": "69db48d0353896a3f13008e2",
    "title": "Harry Potter and the Chamber of Secrets",
    "author": "J.K. Rowling",
    "description": "A dark force threatens Hogwarts as secrets from the"
  }
]
```

The terminal window shows the following commands and output:

```
npm audit fix
ary_Manager\backend> node server.js
[dotenv@17.3.1] injecting env (3) from .env -- tip: override existing env vars with { override: tr
ue }
Server running on port 5000
MongoDB Connected
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

PUT `http://localhost:5000/api/books/69db4e3c558ce893c329cb57` Send

Status: 200 OK Size: 524 Bytes Time: 17 ms

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL

JSON Content Format

```
1 {
2   "genre" : "Fiction"
3 }
```

Response Headers 7 Cookies Results Docs

```
1 {
2   "_id": "69db4e3c558ce893c329cb57",
3   "title": "The Fault in Our Stars",
4   "author": "John Green",
5   "description": "Despite the tumor-shrinking medical miracle that has
6     bought her a few years, Hazel has never been anything but
7     terminal, her final chapter inscribed upon diagnosis. But when a
8     gorgeous plot twist named Augustus Waters suddenly appears at
9     Cancer Kid Support Group, Hazel's story is about to be
10    completely rewritten.",
11   "genre": "Fiction",
12   "createdAt": "2026-04-12T07:48:12.171Z",
13   "updatedAt": "2026-04-12T08:01:48.864Z",
14   "__v": 0
15 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
ary_Manager\backend> node server.js
[dotenv@17.3.1] injecting env (3) from .env -- tip: ⚙ override existing env vars with { override: tr
ue }
Server running on port 5000
MongoDB Connected
```

node backend wsl

DELETE `http://localhost:5000/api/books/69db4e3c558ce893c329cb57` Send

Status: 200 OK Size: 26 Bytes Time: 18 ms

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "genre" : "Fiction"
3 }
```

Response Headers 7 Cookies Results Docs

```
1 {
2   "message": "Book deleted"
3 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
ary_Manager\backend> node server.js
[dotenv@17.3.1] injecting env (3) from .env -- tip: ⚙ override existing env vars with { override: tr
ue }
Server running on port 5000
MongoDB Connected
```

node backend wsl

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title: 7) Perform count and sorting operations on Book Library Manager book records using MongoDB queries.

Page No:

AIM: Perform count and sorting operations on Book Library Manager book records using MongoDB queries.

Description:

This experiment explores MongoDB operations such as count and sort on application data. In Book Library Manager, book records are sorted by creation date and counted to generate dashboard summaries. These operations help organize records and present useful statistics to the user. The project shows practical query handling for library management.

Source Code:

File: backend/src/controllers/bookController.js

```
exports.getAllBooks = async (req, res) => {
  try {
    const { search, category } = req.query;
    let query = {};

    if (search) {
      query.$or = [
        { title: { $regex: search, $options: "i" } },
        { author: { $regex: search, $options: "i" } }
      ];
    }

    if (category) {
      query.category = category;
    }

    const books = await Book.find(query)
      .populate("category", "name")
      .sort({ createdAt: -1 });

    res.json(books);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

exports.getBookStats = async (req, res) => {
  try {
    const totalBooks = await Book.countDocuments();
    const booksByCategory = await Book.aggregate([
      {
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

```
$group: {
  _id: "$category",
  count: { $sum: 1 }
},
{
  $lookup: {
    from: "categories",
    localField: "_id",
    foreignField: "_id",
    as: "categoryInfo"
  },
  {
    $unwind: "$categoryInfo"
  },
  {
    $project: {
      category: "$categoryInfo.name",
      count: 1
    }
  },
  {
    $sort: { count: -1 }
  }
});

res.json({
  totalBooks,
  booksByCategory
});
} catch (error) {
  res.status(500).json({ message: error.message });
}
};
```

File: backend/src/controllers/readingListController.js

```
exports.getMyReadingList = async (req, res) => {
```

```
  try {
```

```
    const readingList = await ReadingList.find({ user: req.user._id })
```

```
    .populate({
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

```
path: "book",
  populate: { path: "category", select: "name" }
})
.sort({ addedAt: -1 });

res.json(readingList);
} catch (error) {
  res.status(500).json({ message: error.message });
}
};
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

OUTPUT:

GET

http://localhost:5000/api/books

Send

Query

Headers²

Auth

Body¹

Tests

Pre Run

HTTP Headers

☐ Raw

☒	Accept	*/*
☒	Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cGE6YWV0IjoiIn0=
☐	header	value

Status: 200 OK Size: 12.53 KB Time: 12 ms

Response

Headers⁷

Cookies

Results

Docs

```
[{"_id": "69db48d0353896a3f13008e1", "title": "Harry Potter and the Sorcerer's Stone", "author": "J.K. Rowling", "description": "A boy discovers he is a wizard and begins his journey at Hogwarts.", "genre": "Fantasy", "__v": 0, "createdAt": "2026-04-12T07:25:04.421Z", "updatedAt": "2026-04-12T07:25:04.421Z"}, {"_id": "69db48d0353896a3f13008e2", "title": "Harry Potter and the Chamber of Secrets", "author": "J.K. Rowling", "description": "A dark force threatens Hogwarts as secrets from the past emerge.", "genre": "Fantasy", "__v": 0, "createdAt": "2026-04-12T07:25:04.422Z", "updatedAt": "2026-04-12T07:25:04.422Z"}]
```

The screenshot displays the MongoDB Atlas web interface. On the left sidebar, the 'Documents' collection under the 'books' database is selected. The main panel shows three documents from the 'documents' collection:

- Document 1:**

```
{
  "_id": ObjectId("69db48d0353896a3f13008e1"),
  "title": "Harry Potter and the Sorcerer's Stone",
  "author": "J.K. Rowling",
  "description": "A boy discovers he is a wizard and begins his journey at Hogwarts.",
  "genre": "Fantasy",
  "__v": NumberInt("0"),
  "createdAt": ISODate("2026-04-12T07:25:04.421Z"),
  "updatedAt": ISODate("2026-04-12T07:25:04.421Z")
}
```
- Document 2:**

```
{
  "_id": ObjectId("69db48d0353896a3f13008e2"),
  "title": "Harry Potter and the Chamber of Secrets",
  "author": "J.K. Rowling",
  "description": "A dark force threatens Hogwarts as secrets from the past emerge.",
  "genre": "Fantasy",
  "__v": NumberInt("0"),
  "createdAt": ISODate("2026-04-12T07:25:04.422Z"),
  "updatedAt": ISODate("2026-04-12T07:25:04.422Z")
}
```
- Document 3:**

```
{
  "_id": ObjectId("69db48d0353896a3f13008e3"),
  "title": "The Hobbit",
  "author": "J.R.R. Tolkien",
  "description": "Bilbo Baggins embarks on a quest to reclaim a stolen treasure guarded by a dragon.",
  "genre": "Fantasy",
  "__v": NumberInt("0"),
  "createdAt": ISODate("2026-04-12T07:25:04.423Z"),
  "updatedAt": ISODate("2026-04-12T07:25:04.423Z")
}
```

The interface includes a top navigation bar with links like 'New Request', 'shelfmaster.books', and 'Preview Screenshot_Execution_Guide_ShelfMaster.md'. A bottom sidebar contains sections for 'PLAYGROUNDS' and 'HELP AND FEEDBACK'.

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title: 8) Develop a styled Book Library Manager signup form and handle user input events in the frontend.

Page No:

AIM: Develop a styled Book Library Manager signup form and handle user input events in the frontend.

Description: This experiment demonstrates a styled form with event handling in the frontend. Book Library Manager uses a React signup form where CSS is applied for layout and appearance. User interactions such as typing and submitting are managed through onChange and onSubmit events. It shows how frontend forms respond dynamically to user input in a web application.

Source Code:

File: frontend/src/pages/Signup.js

```
import React, { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import API from "../services/api";
import "../App.css";
```

```
function Signup() {
  const [formData, setFormData] = useState({
    name: "",
    email: "",
    password: "",
    confirmPassword: ""
  });

  const [loading, setLoading] = useState(false);
  const [message, setMessage] = useState("");
  const [error, setError] = useState("");
  const navigate = useNavigate();

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData((current) => ({
      ...current,
      [name]: value
    }));
  };

  const handleSignup = async (e) => {
    e.preventDefault();
    setMessage("");
    setError("");
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

```
const { name, email, password, confirmPassword } = formData;

if (!name || !email || !password) {
  setError("Please fill in all required fields.");
  return;
}

if (password.length < 6) {
  setError("Password must be at least 6 characters long.");
  return;
}

if (password !== confirmPassword) {
  setError("Passwords do not match.");
  return;
}

try {
  setLoading(true);
  await API.post("/auth/register", {
    name,
    email,
    password
  });
  setMessage("Account created successfully. You can log in now.");
  setTimeout(() => navigate("/"), 700);
} catch (err) {
  setError(err.response?.data?.message || "Signup failed. Please try again.");
} finally {
  setLoading(false);
}
};

return (
  <div className="auth-page"><div className="auth-shell"><div className="hero-panel"><p
className="auth-eyebrow">User Registration</p><h1>Create your Book Library Manager account</h1>
<p>Join our library community and start building your reading list.</p></div><form className="auth-
card" onSubmit={handleSignup}><div className="auth-copy"><p className="auth-eyebrow">Sign
Up</p><h2>Create your account</h2></div><input name="name" placeholder="Full Name" value=
{formData.name} onChange={handleChange}
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

```
    /><inputname="email"type="email"placeholder="Email Address"value=
{formData.email}onChange={handleChange}
    /><inputname="password"type="password"placeholder="Password"value=
{formData.password}onChange={handleChange}
    /><inputname="confirmPassword"type="password"placeholder="Confirm Password"value=
{formData.confirmPassword}onChange={handleChange}
  />
```

```
{error ? <p className="form-message error">{error}</p> : null}
{message ? <p className="form-message success">{message}</p> : null}
```

```
<button type="submit" disabled={loading}>
  {loading ? "Creating account..." : "Sign Up"}
</button><p className="auth-switch">
  Already have an account? <Link to="/">Login</Link></p></form></div></div>
```

```
);
```

```
}
```

```
export default Signup;
```

```
File: frontend/src/App.css
```

```
:root {
  --bg: linear-gradient(135deg, #f6efe3 0%, #e3eef7 100%);
  --panel: rgba(255, 255, 255, 0.92);
  --text: #18324a;
  --accent: #0f7b6c;
  --border: #d4deea;
}
```

```
* {
  box-sizing: border-box;
}
```

```
body {
  margin: 0;
  font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
  color: var(--text);
  background: var(--bg);
}
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

```
input {
  width: 100%;
  border: 1px solid var(--border);
  border-radius: 14px;
  padding: 14px 16px;
  background: white;
}

input:focus {
  outline: 2px solid rgba(15, 123, 108, 0.16);
  border-color: var(--accent);
}

button {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  min-height: 48px;
  padding: 12px 18px;
  border: none;
  border-radius: 14px;
  background: var(--accent);
  color: #fff;
  cursor: pointer;
  font-weight: 700;
}

button:hover {
  background: #0a5f54;
}
```

BVRIT HYDERABAD College of Engineering for Women

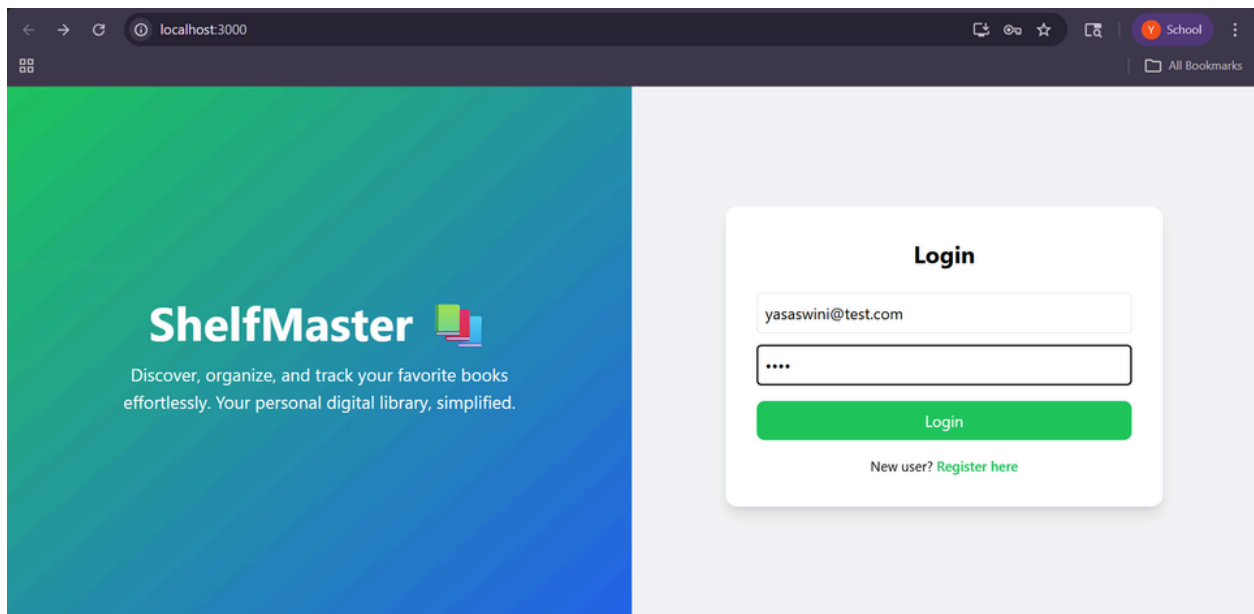
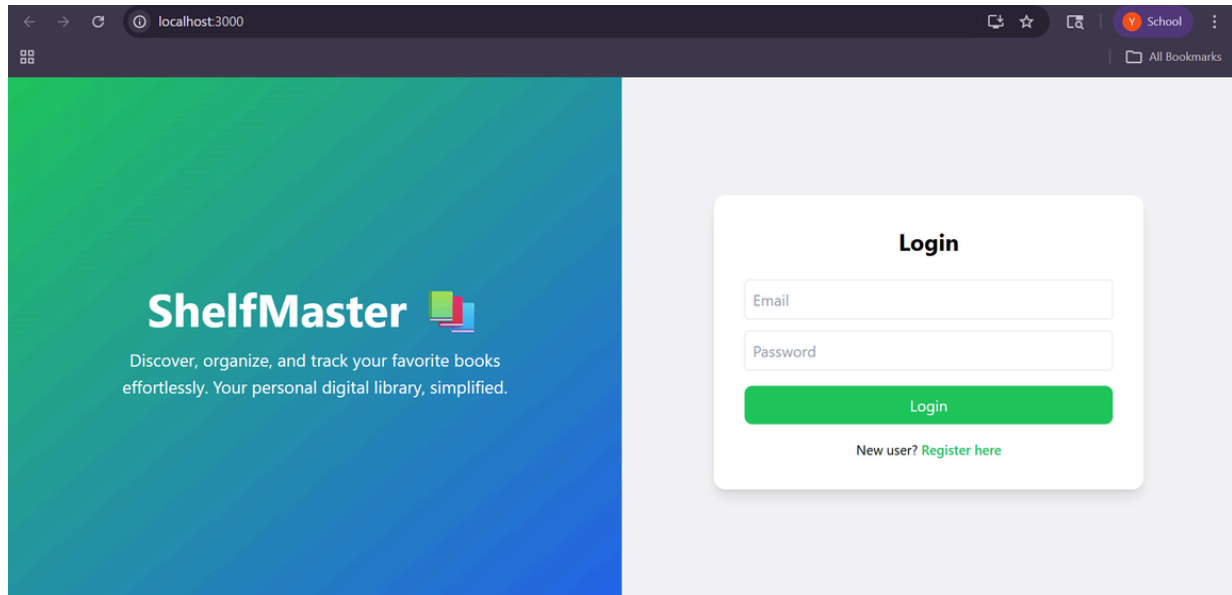
RollNo: 23WH1A0536

Date:

Title:

Page No:

OUTPUT:



BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title: 9) Develop and validate the Book Library Manager user registration form for users and admins.

Page No:

AIM: Develop and validate the Book Library Manager user registration form for users and admins.

Description: This experiment focuses on form validation in a registration system. The Book Library Manager signup and admin signup pages check required fields, password rules, and input matching. The backend also validates data to prevent invalid or duplicate registration requests. It demonstrates both client-side and server-side validation for reliable data entry.

Source Code:

File: frontend/src/pages/AdminSignup.js

```
import React, { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import API from "../services/api";
import "../App.css";
```

```
function AdminSignup() {
  const [formData, setFormData] = useState({
    name: "",
    email: "",
    password: "",
    confirmPassword: "",
    adminSecret: ""
  });

  const [loading, setLoading] = useState(false);
  const [message, setMessage] = useState("");
  const [error, setError] = useState("");
  const navigate = useNavigate();

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData((current) => ({
      ...current,
      [name]: value
    }));
  };

  const handleSignup = async (e) => {
    e.preventDefault();
    setError("");
    setMessage("");
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

```
const { name, email, password, confirmPassword, adminSecret } = formData;

if (!name || !email || !password || !confirmPassword || !adminSecret) {
  setError("Please fill in all required fields.");
  return;
}

if (password.length < 6) {
  setError("Password must be at least 6 characters long.");
  return;
}

if (password !== confirmPassword) {
  setError("Passwords do not match.");
  return;
}

try {
  setLoading(true);
  await API.post("/auth/register-admin", {
    name,
    email,
    password,
    adminSecret
  });
  setMessage("Admin account created successfully. You can log in now.");
  setTimeout(() => navigate("/admin/login"), 700);
} catch (err) {
  setError(err.response?.data?.message || "Admin signup failed. Please try again.");
} finally {
  setLoading(false);
}
};

return (
  <div className="auth-page"><div className="auth-shell"><form className="auth-card" onSubmit={
handleSignup}><h2>Admin Registration</h2><input name="name" placeholder="Full Name" value=
{formData.name} onChange={handleChange}>
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

```
const result = await createUser({
  name,
  email,
  password,
  role: "admin"
});

if (result.error) {
  return res.status(result.error.status).json({ message: result.error.message });
}

res.status(201).json({
  message: "Admin account created successfully",
  user: sanitizeUser(result.user)
});
} catch (err) {
  res.status(400).json({ message: err.message });
}
};
```

BVRIT HYDERABAD College of Engineering for Women

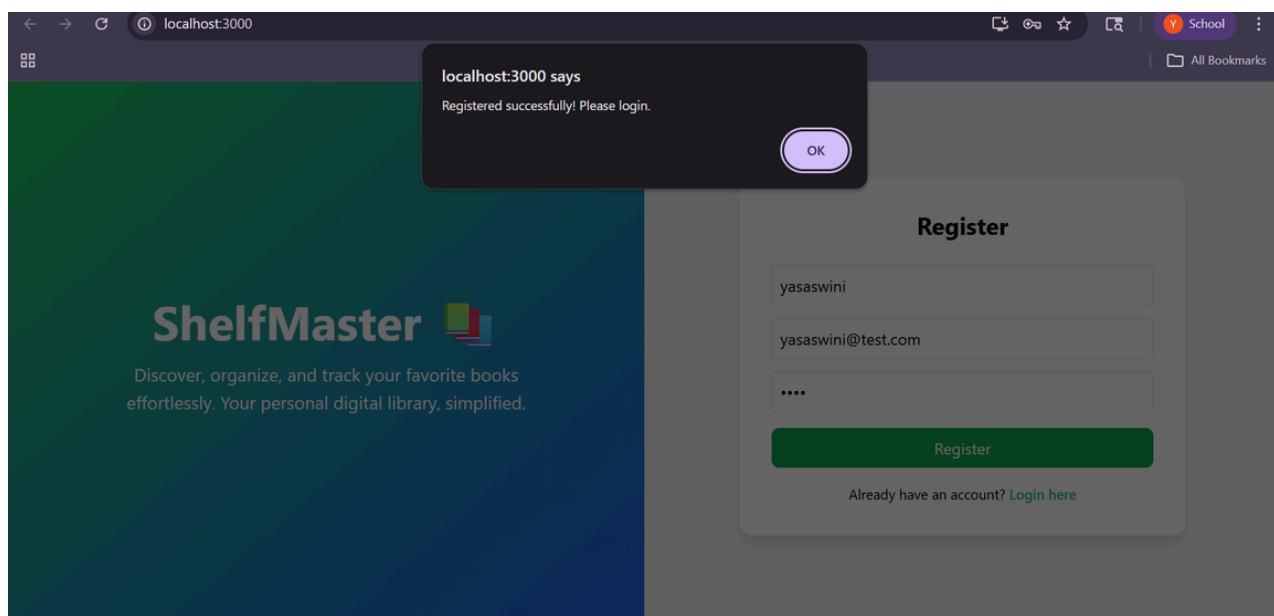
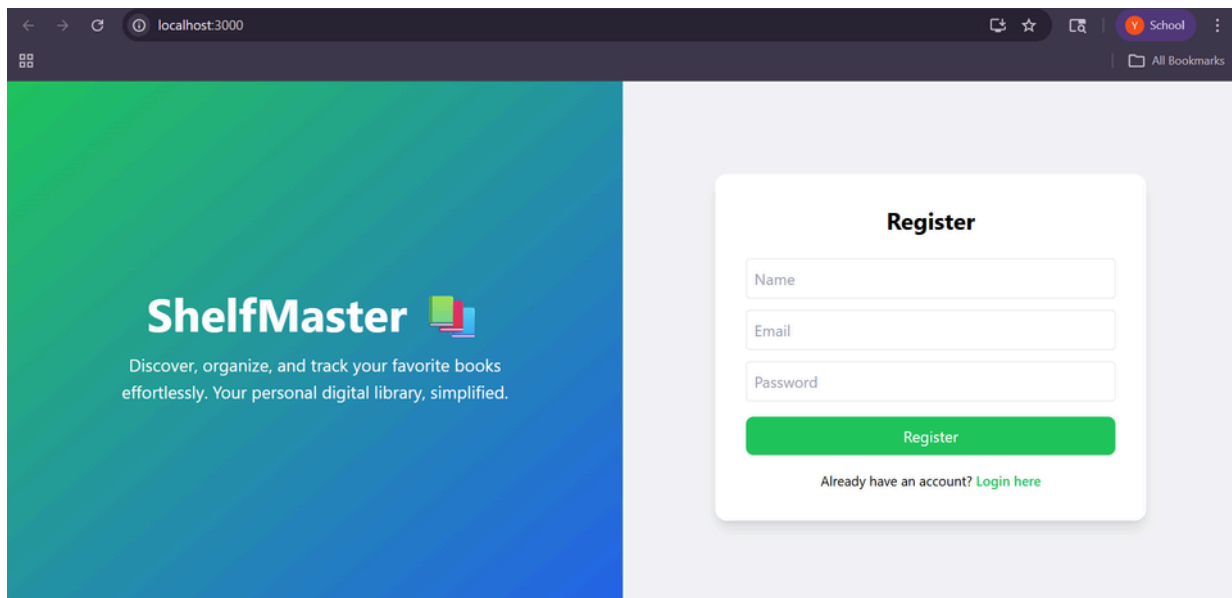
RollNo: 23WH1A0536

Date:

Title:

Page No:

OUTPUT:



BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title: 10) Fetch and display Book Library Manager JSON data from the backend using frontend service calls.

Page No:

AIM: Fetch and display Book Library Manager JSON data from the backend using frontend service calls.

Description:

This experiment shows how a frontend application fetches JSON data from a server. In Book Library Manager, React components use Axios services to request dashboard and book data from backend APIs. The received JSON response is processed and displayed in the user interface. It explains how frontend and backend communicate to build a data-driven application.

Source Code:

File: frontend/src/services/api.js

```
import axios from "axios";

const API = axios.create({
  baseURL: "http://localhost:5000/api"
});

API.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export default API;
```

File: frontend/src/pages/Dashboard.js

```
import React, { useEffect, useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import API from "../services/api";
import { clearSession, getStoredUser } from "../services/auth";
import "../App.css";

function Dashboard() {
  const navigate = useNavigate();
  const [user, setUser] = useState(null);
  const [stats, setStats] = useState({
    totalBooks: 0,
    totalCategories: 0,
    myReadingList: 0
  });
}
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

```
useEffect(() => {
  const storedUser = getStoredUser();
  if (!storedUser) {
    navigate("/");
    return;
  }
  setUser(storedUser);
}, [navigate]);

useEffect(() => {
  if (!user) {
    return;
  }
}, [user]);

const loadStats = async () => {
  try {
    const [booksRes, categoriesRes, readingListRes] = await Promise.all([
      API.get("/books"),
      API.get("/categories"),
      API.get("/reading-list")
    ]);

    setStats({
      totalBooks: booksRes.data.length,
      totalCategories: categoriesRes.data.length,
      myReadingList: readingListRes.data.length
    });
  } catch (error) {
    if (error.response?.status === 401) {
      clearSession();
      navigate("/");
    }
  }
};

loadStats();
}, [navigate, user]);

const handleLogout = () => {
  clearSession();
  navigate("/");
};
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

```
if (!user) {
  return null;
}

return (
  <div className="dashboard-page"><section className="dashboard-header"><div><p
className="auth-eyebrow">Dashboard</p><h1>Welcome, {user.name}</h1><p
className="dashboard-subtitle">
  {user.role === "admin"
    ? "Manage books, categories, and users from your admin workspace."
    : "Browse books, build your reading list, and discover new titles."}
</p></div><div className="top-actions">
  {user.role === "admin" ? (
    <Link className="button-link" to="/admin/books">
      Manage Books
    </Link>
  ) : (
    <><Link className="button-link" to="/books">
      Browse Books
    </Link><Link className="button-link secondary-button-link" to="/reading-list">
      My Reading List
    </Link></>
  )}
  <button className="secondary-button" onClick={handleLogout}>
    Logout
  </button></div></section><section className="summary-grid"><div className="summary-card">
<span>Total Books</span><strong>{stats.totalBooks}</strong></div><div className="summary-card">
<span>Categories</span><strong>{stats.totalCategories}</strong></div><div className="summary-
card"><span>My Reading List</span><strong>{stats.myReadingList}</strong></div></section>
</div>
);
}
```

export default Dashboard;

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

OUTPUT:

The screenshot shows the VS Code interface with a REST client extension. The top panel displays a REST client request and its response. The request is a GET to `http://localhost:3000/books`. The response is an HTML document with a description of a web application. The bottom panel shows the terminal output of the `npm start` command, indicating a successful webpack build.

REST Client Request:

```
GET http://localhost:3000/books
```

REST Client Response:

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="utf-8" />
5     <link rel="icon" href="/favicon.ico" />
6     <meta name="viewport" content="width=device-width, initial
  -scale=1" />
7     <meta name="theme-color" content="#000000" />
8     <meta
9       name="description"
10      content="Web site created using create-react-app" />
11     <link rel="apple-touch-icon" href="/logo192.png" />
12   <!--
13     manifest.json provides metadata used when your web app is
      installed on a
```

Terminal Output:

```
PS C:\Users\Yasaswini\OneDrive\Desktop\Book_Library_Manager\frontend> npm start

Note that the development build is not optimized.
To create a production build, use npm run build.

webpack compiled successfully
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title: 11) Develop the Book Library Manager web application to manage book and user information using Express and React.

Page No:

AIM: Develop the Book Library Manager web application to manage book and user information using Express and React.

Description: This experiment develops a web application for managing book-related information using Express and a frontend framework. Book Library Manager supports registration, login, dashboard access, and book data retrieval through connected backend and frontend modules. The Express server handles routes and APIs, while React provides the interactive user interface. It demonstrates a complete full-stack application structure for library management.

Source Code:

File: backend/server.js

```
const app = express();

connectDB();

app.use(cors());
app.use(express.json());

app.use("/api/users", userRoutes);
app.use("/api/books", bookRoutes);
app.use("/api/categories", categoryRoutes);
app.use("/api/reading-list", readingListRoutes);
app.use("/api/auth", authRoutes);

app.get("/", (req, res) => {
  res.send("Book Library Manager Backend Running");
});
```

File: backend/src/routes/categoryRoutes.js

```
const express = require("express");
const {
  getAllCategories,
  getCategoryById,
  createCategory,
  updateCategory,
  deleteCategory
} = require("../controllers/categoryController");
const { protect, authorize } = require("../middleware/authMiddleware");

const router = express.Router();

router.get("/", getAllCategories);
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

```
router.get("/:id", getCategoryById);
router.post("/", protect, authorize("admin"), createCategory);
router.put("/:id", protect, authorize("admin"), updateCategory);
router.delete("/:id", protect, authorize("admin"), deleteCategory);
```

```
module.exports = router;
```

File: backend/src/routes/userRoutes.js

```
const router = express.Router();
```

```
router.get("/", protect, authorize("admin"), async (req, res) => {
  try {
    const users = await User.find().select("-password").sort({ createdAt: -1 });
    res.json(users);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});
```

```
module.exports = router;
```

File: frontend/src/pages/Signup.js

```
const handleSignup = async (e) => {
  e.preventDefault();
  setMessage("");
  setError("");
```

```
const { name, email, password, confirmPassword } = formData;
```

```
if (!name || !email || !password) {
  setError("Please fill in all required fields.");
  return;
}
```

```
if (password !== confirmPassword) {
  setError("Passwords do not match.");
  return;
}
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

```
await API.post("/auth/register", {
  name,
  email,
  password
});
```

```
setMessage("Account created successfully. You can log in now.");
};
```

File: frontend/src/pages/Dashboard.js

```
const loadStats = async () => {
  try {
    const booksRes = await API.get("/books");
    const categoriesRes = await API.get("/categories");
    const readingListRes = await API.get("/reading-list");

    setStats({
      totalBooks: booksRes.data.length,
      totalCategories: categoriesRes.data.length,
      myReadingList: readingListRes.data.length
    });
  } catch (error) {
    if (error.response?.status === 401) {
      clearSession();
      navigate("/");
    }
  }
};
```

BVRIT HYDERABAD College of Engineering for Women

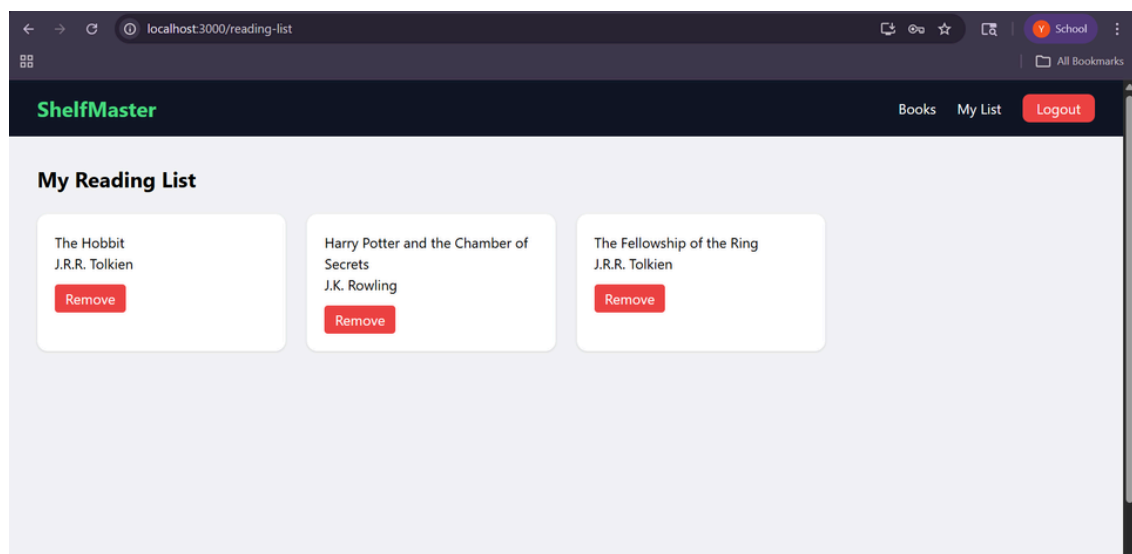
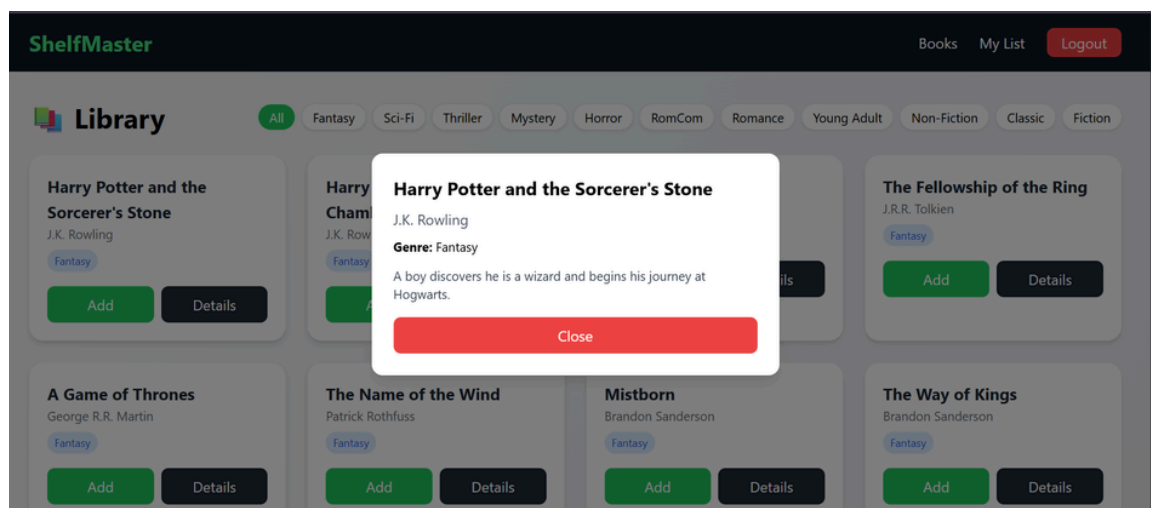
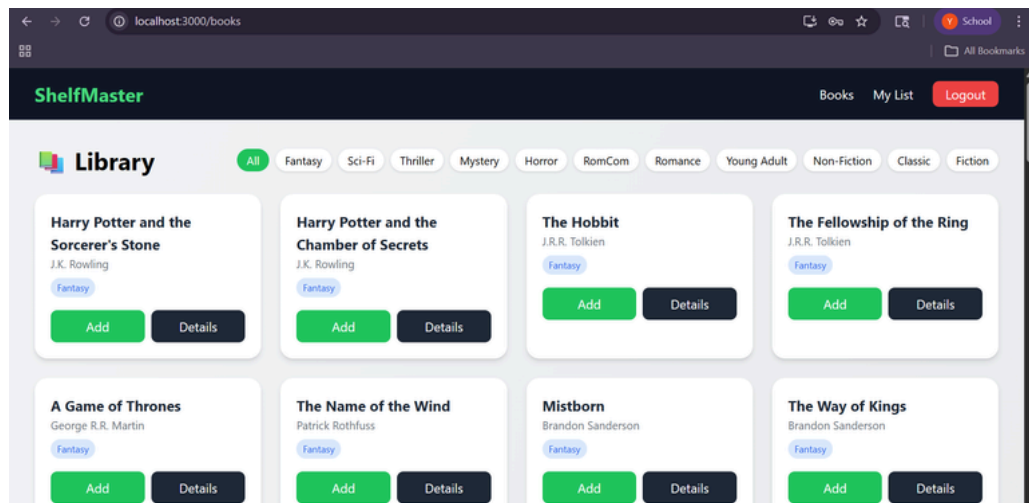
RollNo: 23WH1A0536

Date:

Title:

Page No:

OUTPUT:



BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title: 12)Implement an approval decision workflow in React for reviewing Book Library Manager reading list requests.

Page No:

AIM: Implement an approval decision workflow in React for reviewing Book Library Manager reading list requests.

Description:

This experiment presents a React-based decision workflow similar to a voting application. In Book Library Manager, users can add or remove books from their reading list from the book details page. Each decision updates the status of the reading list in the database and interface. It shows how action-based state changes can be implemented in a React application.

Source Code:

File: frontend/src/pages/BookDetails.js

```
import React, { useState, useEffect } from "react";
import { useParams, useNavigate } from "react-router-dom";
import API from "../services/api";
import { getStoredUser } from "../services/auth";
```

```
function BookDetails() {
  const { id } = useParams();
  const navigate = useNavigate();
  const [book, setBook] = useState(null);
  const [inReadingList, setInReadingList] = useState(false);
  const [loading, setLoading] = useState(true);
  const user = getStoredUser();
```

```
  useEffect(() => {
    fetchBook();
    if (user) {
      checkReadingList();
    }
  }, [id]);
```

```
  const fetchBook = async () => {
    try {
      const res = await API.get(`/books/${id}`);
      setBook(res.data);
    } catch (error) {
      console.error(error);
    } finally {
      setLoading(false);
    }
  };
};
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

```
const checkReadingList = async () => {
  try {
    const res = await API.get("/reading-list");
    setInReadingList(res.data.some((item) => item.book._id === id));
  } catch (error) {
    console.error(error);
  }
};

const handleAddToList = async () => {
  try {
    await API.post("/reading-list", { bookId: id });
    setInReadingList(true);
    alert("Added to reading list");
  } catch (error) {
    alert(error.response?.data?.message || "Failed to add to reading list");
  }
};

const handleRemoveFromList = async () => {
  try {
    await API.delete(`/reading-list/${id}`);
    setInReadingList(false);
    alert("Removed from reading list");
  } catch (error) {
    alert(error.response?.data?.message || "Failed to remove from reading list");
  }
};

if (loading) return <div>Loading...</div>;
if (!book) return <div>Book not found</div>;

return (
  <div className="book-details-page"><button onClick={() => navigate(-1)}>Back</button><h1>
{book.title}</h1><h2>by {book.author}</h2><p><strong>Category:</strong> {book.category?.name}</p>
<p><strong>Published:</strong> {book.publishedYear}</p><p><strong>ISBN:</strong> {book.isbn}</p>
<p>{book.description}</p>
  </div>
);
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

```
{user && user.role === "user" && (  
  <div className="action-row">  
    {inReadingList ? (  
      <button onClick={handleRemoveFromList}>  
        Remove from Reading List  
      </button>  
    ) : (  
      <button onClick={handleAddToList}>  
        Add to Reading List  
      </button>  
    )}  
  </div>  
)}  
</div>  
);  
}
```

export default BookDetails;

File: backend/src/controllers/readingListController.js

```
const ReadingList = require("../models/ReadingList");
```

```
exports.addToReadingList = async (req, res) => {
```

```
  try {
```

```
    const { bookId } = req.body;
```

```
    const existing = await ReadingList.findOne({
```

```
      user: req.user._id,
```

```
      book: bookId
```

```
    });
```

```
    if (existing) {
```

```
      return res.status(400).json({ message: "Book already in reading list" });
```

```
    }
```

```
    const readingListItem = new ReadingList({
```

```
      user: req.user._id,
```

```
      book: bookId
```

```
    });
```

```
    await readingListItem.save();
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

```
const populated = await ReadingList.findById(readingListItem._id)
    .populate({
      path: "book",
      populate: { path: "category", select: "name" }
    });

res.status(201).json(populated);
} catch (error) {
  res.status(500).json({ message: error.message });
}
};

exports.removeFromReadingList = async (req, res) => {
  try {
    const result = await ReadingList.findOneAndDelete({
      user: req.user._id,
      book: req.params.bookId
    });

    if (!result) {
      return res.status(404).json({ message: "Book not found in reading list" });
    }

    res.json({ message: "Book removed from reading list" });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

BVRIT HYDERABAD College of Engineering for Women

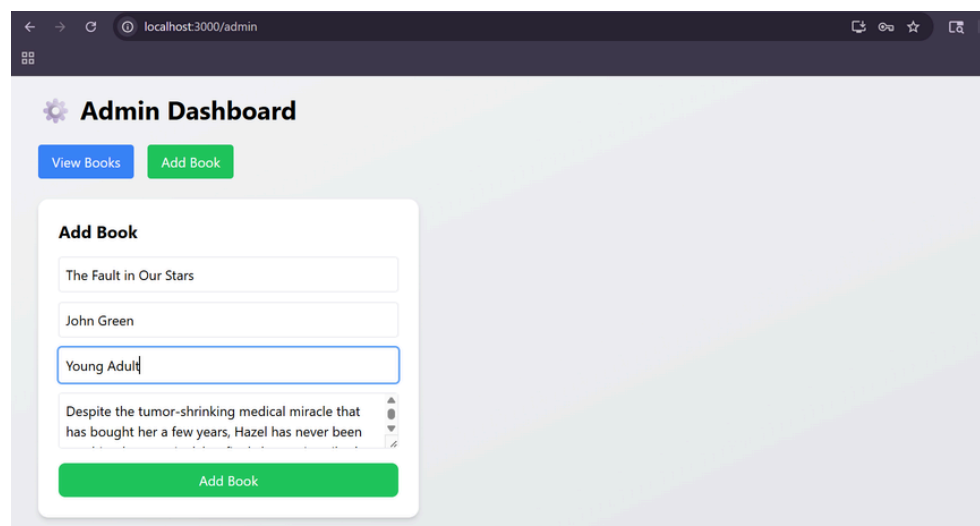
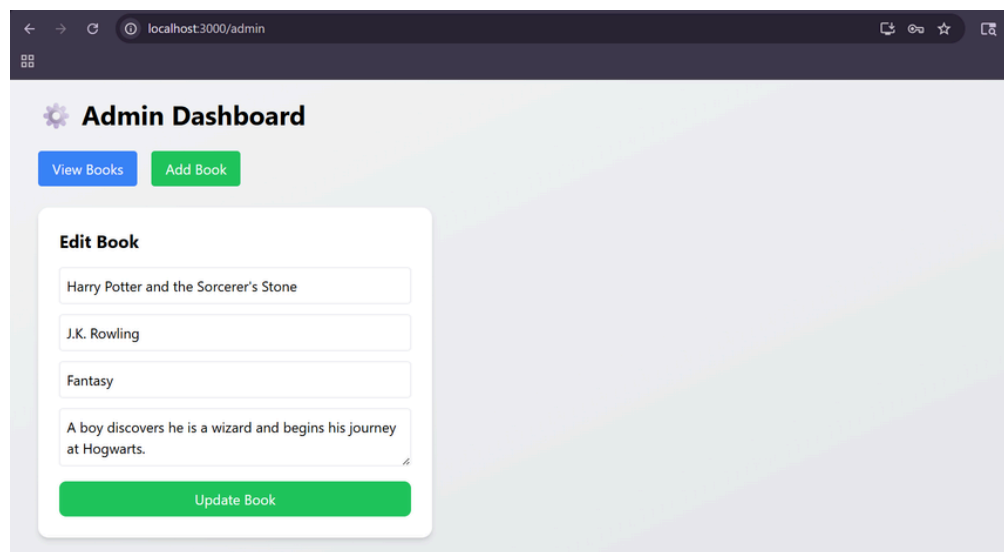
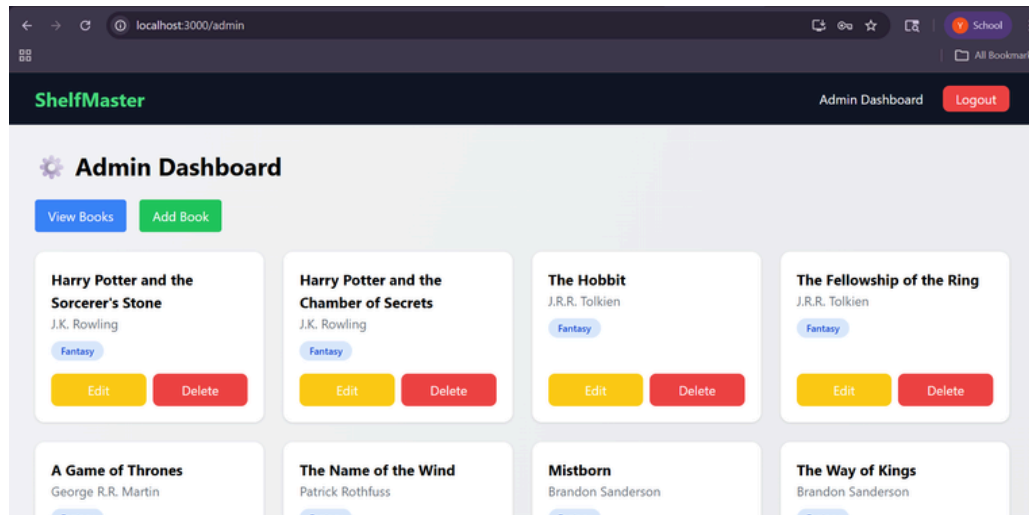
RollNo: 23WH1A0536

Date:

Title:

Page No:

OUTPUT:



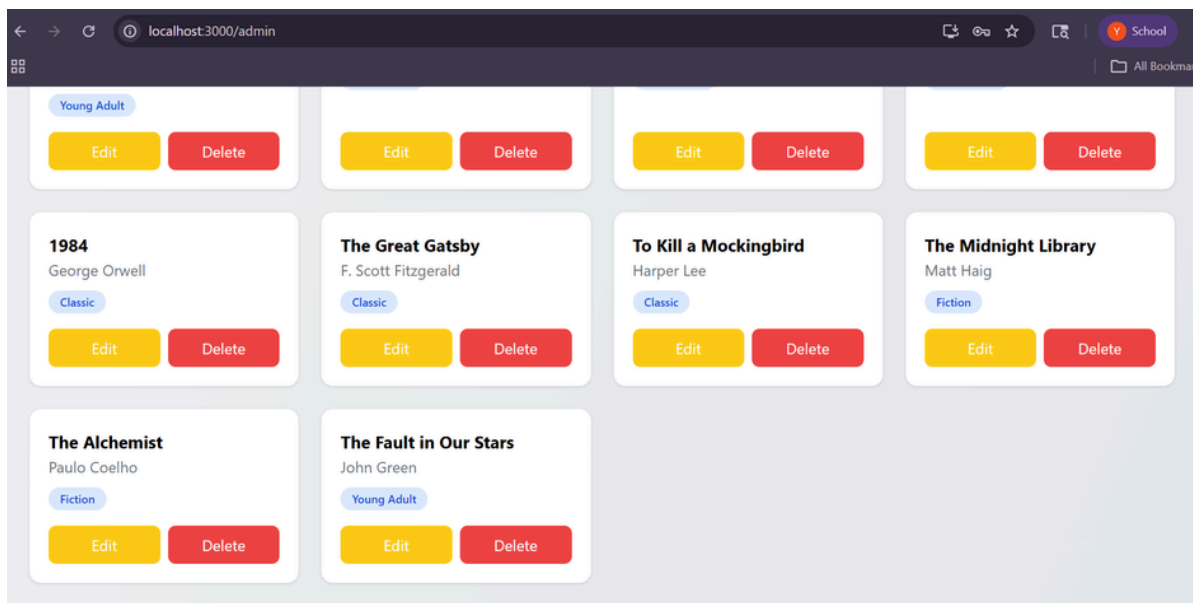
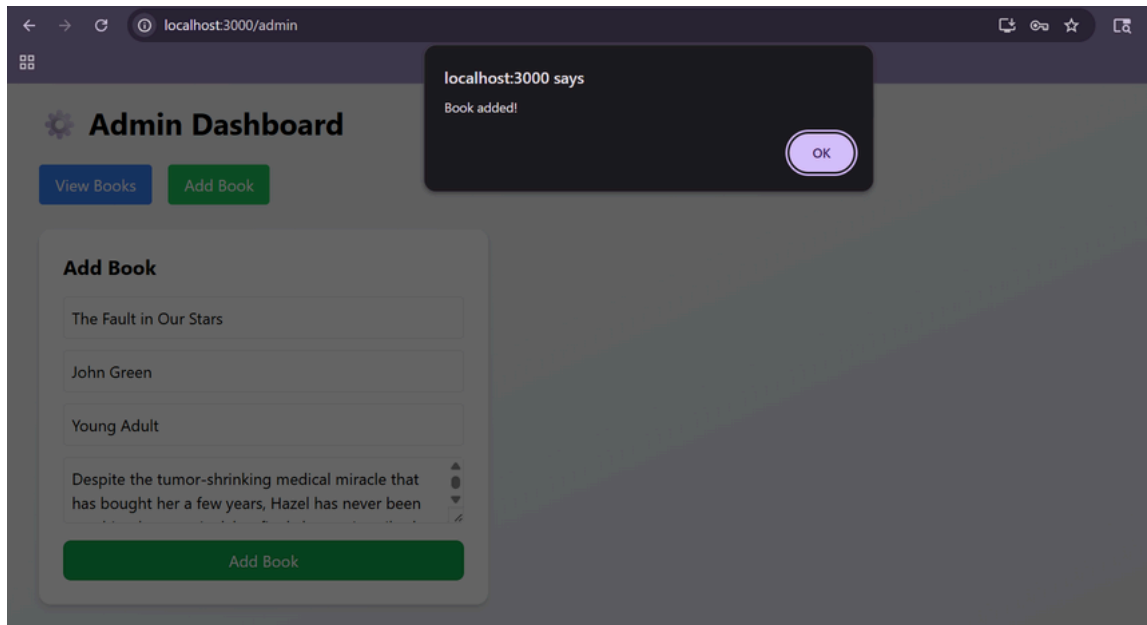
BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:



BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title: 13)Develop the Book Library Manager reading list management system using React and Node.js.

Page No:

AIM: Develop the Book Library Manager reading list management system using React and Node.js.

Description:

This experiment develops a reading list management system using React, which directly matches the Book Library Manager project. Users can add books to their personal reading list, view their saved books, and remove books from the list. The backend stores each reading list entry in MongoDB with user and book references. It represents the core functionality and main objective of the entire project.

Source Code:

File: backend/src/models/ReadingList.js

```
const mongoose = require("mongoose");

const readingListSchema = new mongoose.Schema({
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true
  },
  book: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Book",
    required: true
  },
  addedAt: {
    type: Date,
    default: Date.now
  }
});

readingListSchema.index({ user: 1, book: 1 }, { unique: true });

module.exports = mongoose.model("ReadingList", readingListSchema);
```

File: backend/src/controllers/readingListController.js

```
exports.getMyReadingList = async (req, res) => {
  try {
    const readingList = await ReadingList.find({ user: req.user._id })
      .populate({
        path: "book",
        populate: { path: "category", select: "name" }
      })
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

```
.sort({ addedAt: -1 });

res.json(readingList);
} catch (error) {
  res.status(500).json({ message: error.message });
}
};

exports.addToReadingList = async (req, res) => {
  try {
    const { bookId } = req.body;

    if (!bookId) {
      return res.status(400).json({ message: "Book ID is required" });
    }

    const existing = await ReadingList.findOne({
      user: req.user._id,
      book: bookId
    });

    if (existing) {
      return res.status(400).json({ message: "Book already in reading list" });
    }

    const readingListItem = new ReadingList({
      user: req.user._id,
      book: bookId
    });
    await readingListItem.save();

    const populated = await ReadingList.findById(readingListItem._id)
      .populate({
        path: "book",
        populate: { path: "category", select: "name" }
      });

    res.status(201).json(populated);
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
};
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

File: frontend/src/pages/ReadingList.js

```
import React, { useState, useEffect } from "react";
import { Link, useNavigate } from "react-router-dom";
import API from "../services/api";
import { getStoredUser } from "../services/auth";

function ReadingList() {
  const [readingList, setReadingList] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");
  const navigate = useNavigate();

  useEffect(() => {
    const user = getStoredUser();
    if (!user || user.role !== "admin") {
      navigate("/");
      return;
    }

    const loadReadingList = async () => {
      try {
        const res = await API.get("/reading-list");
        setReadingList(res.data);
      } catch (err) {
        setError(err.response?.data?.message || "Could not load reading list.");
      } finally {
        setLoading(false);
      }
    };

    loadReadingList();
  }, [navigate]);

  const handleRemove = async (bookId) => {
    try {
      await API.delete(`/reading-list/${bookId}`);
      setReadingList(readingList.filter((item) => item.book._id !== bookId));
    } catch (err) {
      alert("Failed to remove book from reading list");
    }
  };
}
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

```
return (  
  <div className="reading-list-page"><section className="dashboard-header"><div><p  
className="auth-eyebrow">My Reading List</p><h1>Your Saved Books</h1><p  
className="dashboard-subtitle">  
  Review your saved books and manage your personal library.  
  </p></div><div className="top-actions"><Link className="button-link secondary-button-link"  
to="/dashboard">  
  Dashboard  
  </Link><Link className="button-link" to="/books">  
  Browse Books  
  </Link></div></section>  
  
  {loading ? <div className="panel-card">Loading reading list...</div> : null}  
  {error ? <div className="panel-card error-box">{error}</div> : null}  
  
  {!loading && !error ? (  
    <div className="list-grid">  
      {readingList.length === 0 ? (  
        <div className="panel-card">  
          No books in your reading list yet. Start browsing to add books!  
        </div>  
      ) : (  
        readingList.map((item) => (  
          <div className="panel-card" key={item._id}><div className="card-header"><h3>  
{item.book.title}</h3><span className="category-badge">{item.book.category?.name}</span></div>  
<p><strong>Author:</strong> {item.book.author}</p><p><strong>Published:</strong>  
{item.book.publishedYear}</p><p>{item.book.description}</p><p><strong>Added:</strong> {new  
Date(item.addedAt).toLocaleDateString()}</p><div className="action-row"><Link to=  
{`/books/${item.book._id}`}>View Details</Link><button onClick={() => handleRemove(item.book._id)}>  
  Remove  
</button></div></div>  
        </div>  
      ) : null}  
    </div>  
  ) : null}  
);  
}
```

export default ReadingList;

BVRIT HYDERABAD College of Engineering for Women

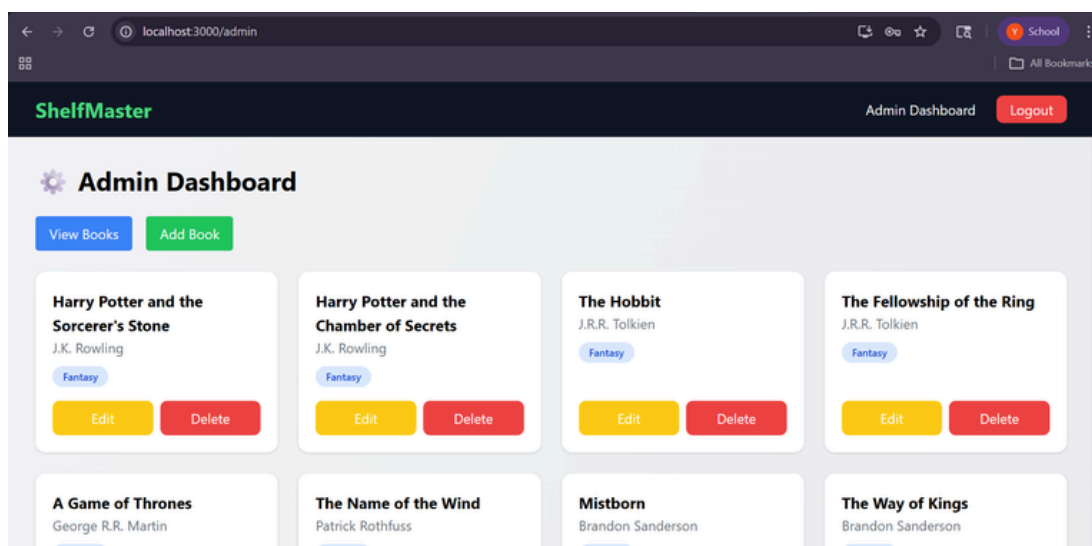
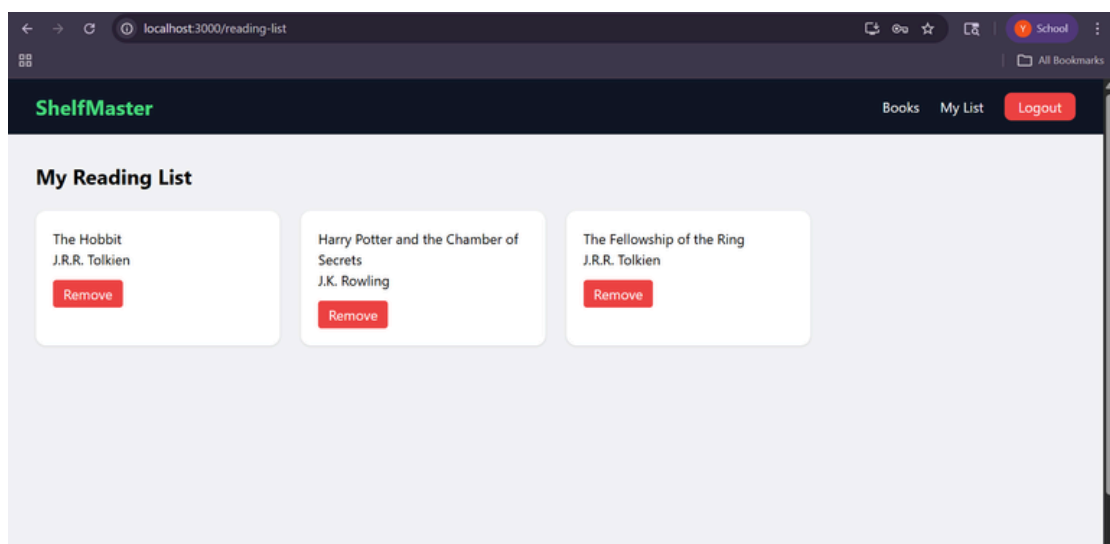
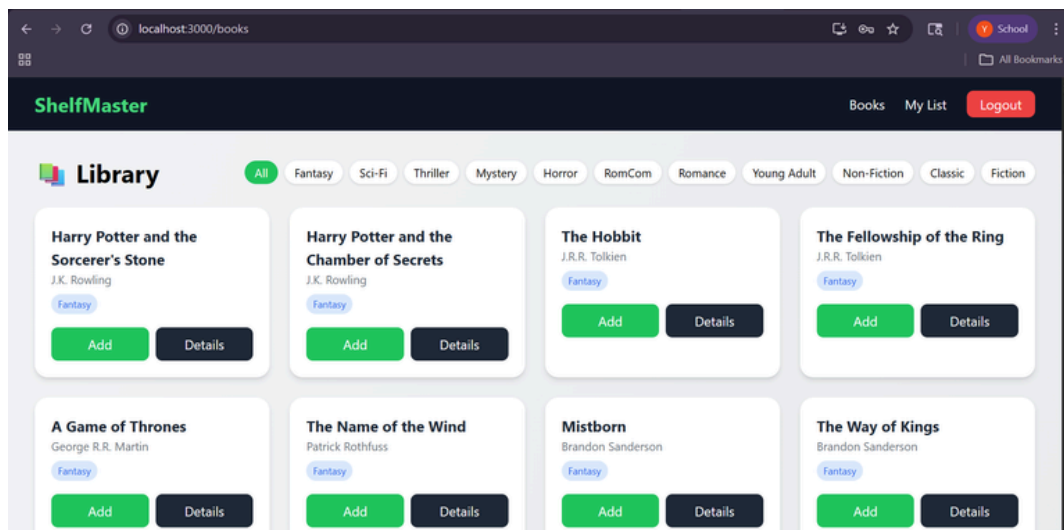
RollNo: 23WH1A0536

Date:

Title:

Page No:

OUTPUT:



BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title: 14) Build the Book Library Manager React application using reusable components and routing between pages.

Page No:

AIM: Build the Book Library Manager React application using reusable components and routing between pages.

Description: This experiment demonstrates the use of React components and routing across multiple web pages. Book Library Manager includes separate pages for login, signup, dashboard, book browsing, book details, reading list, and admin management. React Router is used to navigate between pages, and protected routes restrict access to authorized users. It explains how component-based design and routing improve the structure of a multi-page frontend application.

Source Code:

File: frontend/src/App.js

```
import React from "react";
import { BrowserRouter, Route, Routes } from "react-router-dom";
import ProtectedRoute from "../components/ProtectedRoute";
import AdminLogin from "../pages/AdminLogin";
import AdminSignup from "../pages/AdminSignup";
import BookDetails from "../pages/BookDetails";
import BrowseBooks from "../pages/BrowseBooks";
import Dashboard from "../pages/Dashboard";
import Login from "../pages/Login";
import ManageBooks from "../pages/ManageBooks";
import ReadingList from "../pages/ReadingList";
import ResetPassword from "../pages/ResetPassword";
import Signup from "../pages/Signup";
function App() {
  return (
    <BrowserRouter><Routes><Route path="/" element={<Login />} />
      <Route path="/signup" element={<Signup />} />
      <Route path="/reset-password" element={<ResetPassword />} />
      <Route path="/admin/login" element={<AdminLogin />} />
      <Route path="/admin/signup" element={<AdminSignup />} />
      <Route element={<ProtectedRoute allowedRoles={["user", "admin"]} />>
        <Route path="/dashboard" element={<Dashboard />} />
        <Route path="/books" element={<BrowseBooks />} />
        <Route path="/books/:id" element={<BookDetails />} />
      </Route><Route element={<ProtectedRoute allowedRoles={["user"]} />>
        <Route path="/reading-list" element={<ReadingList />} />
      </Route><Route element={<ProtectedRoute allowedRoles={["admin"]} />>
        <Route path="/admin/books" element={<ManageBooks />} />
      </Route></Routes></BrowserRouter>
  );
}
export default App;
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

File: frontend/src/components/ProtectedRoute.js

```
import React from "react";
import { Navigate, Outlet } from "react-router-dom";
import { getStoredUser } from "../services/auth";

function ProtectedRoute({ allowedRoles }) {
  const user = getStoredUser();

  if (!user) {
    return <Navigate to="/" replace />;
  }

  if (allowedRoles && !allowedRoles.includes(user.role)) {
    return <Navigate to="/dashboard" replace />;
  }

  return <Outlet />;
}

export default ProtectedRoute;
```

File: frontend/src/pages/Login.js

```
import React, { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import API from "../services/api";
import { saveSession } from "../services/auth";

function Login() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");
  const navigate = useNavigate();

  const handleLogin = async (e) => {
    e.preventDefault();
    setError("");

    if (!email || !password) {
      setError("Email and password are required");
      return;
    }
  }
}
```

BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

```
try {
  setLoading(true);
  const res = await API.post("/auth/login", { email, password });
  saveSession(res.data.token, res.data.user);
  navigate("/dashboard");
} catch (err) {
  setError(err.response?.data?.message || "Login failed");
} finally {
  setLoading(false);
}
};

return (
  <div className="auth-page"><div className="auth-shell"><div className="hero-panel"><p
className="auth-eyebrow">Book Library Manager</p><h1>Simplify book management for readers and
admins</h1></div><form className="auth-card" onSubmit={handleLogin}>
<input type="email" placeholder="Email Address" value={email} onChange={(e) =>
setEmail(e.target.value)}
/>
<input type="password" placeholder="Password" value={password} onChange={(e) =>
setPassword(e.target.value)}
/>
{error ? <p className="form-message error">{error}</p> : null}
<button type="submit" disabled={loading}>
  {loading ? "Signing in..." : "Login"}
</button><p className="auth-switch">
  Don't have an account? <Link to="/signup">Sign up</Link></p></form></div></div>
);
}

export default Login;
File: frontend/src/services/auth.js
export const saveSession = (token, user) => {
  localStorage.setItem("token", token);
  localStorage.setItem("user", JSON.stringify(user));
};

export const getStoredUser = () => {
  const user = localStorage.getItem("user");
  return user ? JSON.parse(user) : null;
};

export const clearSession = () => {
  localStorage.removeItem("token");
  localStorage.removeItem("user");
};
};
```

BVRIT HYDERABAD College of Engineering for Women

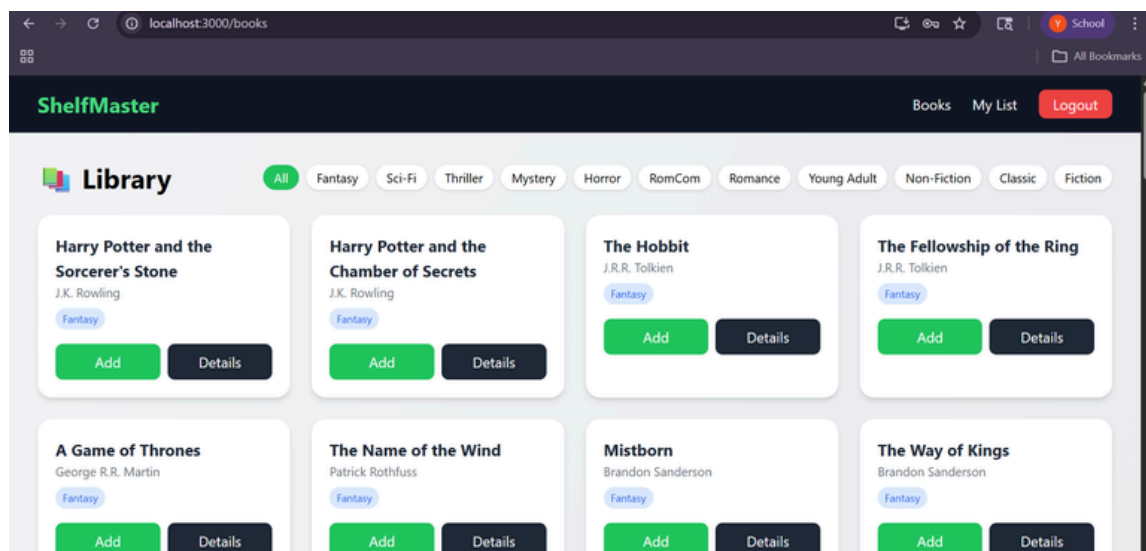
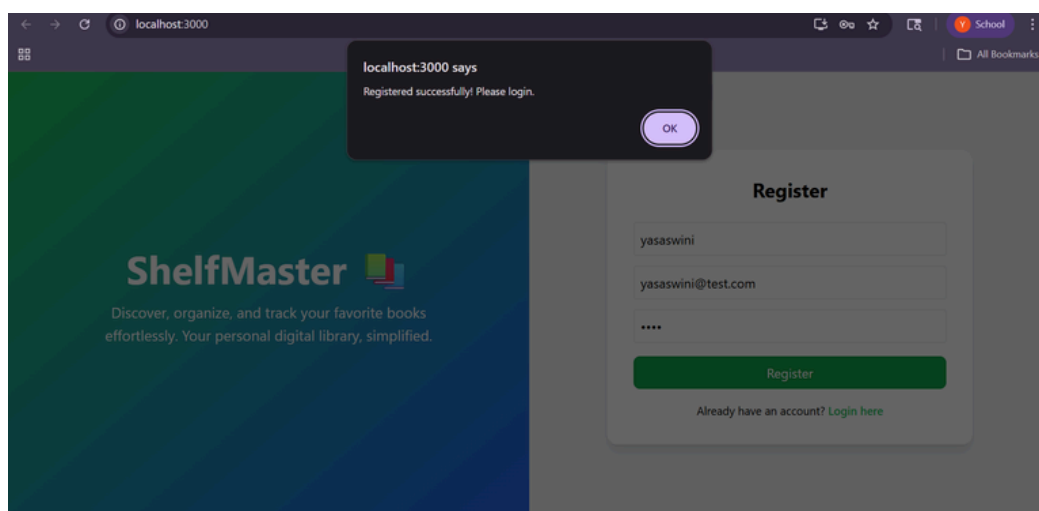
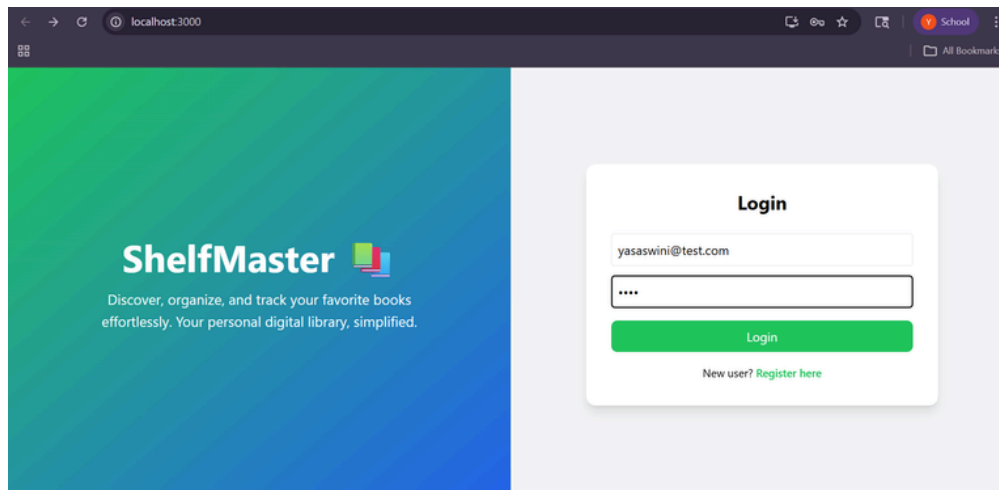
RollNo: 23WH1A0536

Date:

Title:

Page No:

OUTPUT:



BVRIT HYDERABAD College of Engineering for Women

RollNo: 23WH1A0536

Date:

Title:

Page No:

